

StudyPlanner — Technical Documentation

Version	1.0
Last updated	2026-08-25
Live app	clinquant-bonbon-7a0dd6.netlify.app
Repository	study management website (branch main)

1. Purpose and Scope

StudyPlanner is a web-based study management application for individual students. It centers on a single core principle:

The timer is the source of truth for everything. Every statistic, streak, and time-tracking number in the app is derived from actual recorded `study_sessions`, not from self-reported estimates.

This document describes the system's functional requirements and primary use cases, and summarizes the architecture and data model that implement them. It is intended for developers maintaining or extending the codebase.

2. System Overview

Layer	Technology
Frontend	React 19, Vite, TypeScript, Tailwind CSS v4
Server state	TanStack Query (React Query) v5
Backend	Supabase (PostgreSQL, Auth, Row Level Security)
Charts	Recharts
Offline/installable	PWA (manifest, service worker, custom icons)
Hosting	Netlify (frontend) + Supabase (backend), GitHub for source control

2.1 Architecture Principles

- **RLS-enforced privacy, not app-code discipline.** Access control for cross-user data (friends) is enforced at the database layer via PostgreSQL Row Level Security policies and `SECURITY DEFINER` functions — a bug in the frontend cannot leak private data, because the database itself refuses the query.
- **Derived data over duplicated state.** Aggregates such as `daily_stats` and `streaks` are recomputed by Postgres triggers whenever `study_sessions` changes, rather than being written directly by the client. This keeps the timer as the single source of truth.
- **Timestamp-based timer, not tick-counted.** The Pomodoro timer persists `phaseStartedAt` / `pausedAt` / `accumulatedPauseMs` to `localStorage` and recomputes elapsed time from wall-clock timestamps on every render, so it cannot drift or reset when a tab is backgrounded or the page is refreshed.

2.2 Actors

Actor	Description
Student (primary user)	A signed-in user who tracks their own study activity, todos, exams, and questions.
Friend	Another Student who has an accepted, mutual friendship with the primary user. Can view only the primary user's aggregate time/streak numbers (and screen time, if explicitly shared) — never subject, todo, exam, or question content.
System (triggers/scheduled logic)	Server-side Postgres functions that recompute daily stats, streaks, and spaced-repetition intervals automatically in response to data changes.

3. Functional Requirements

Each requirement has a unique ID for traceability. Priority: **M** = Must have (implemented, core to the spec), **S** = Should have (implemented, enhancement).

3.1 Authentication & Account (FR-1x)

ID	Requirement	Priority
FR-1.1	The system shall let a user sign up and sign in via email magic link (passwordless).	M
FR-1.2	On first sign-up, the system shall automatically create a <code>profiles</code> row (with a unique 8-character friend code), a default <code>user_settings</code> row, a <code>streaks</code> row, and a <code>study_presence</code> row.	M
FR-1.3	The system shall protect all app routes except <code>/login</code> behind an authenticated-session check, redirecting unauthenticated users to <code>/login</code> .	M

3.2 Pomodoro Timer (FR-2x)

ID	Requirement	Priority
FR-2.1	The system shall run a focus/break Pomodoro timer with durations configurable in Settings (default 25 min focus / 5 min break).	M
FR-2.2	The user shall be able to optionally link a running timer session to a subject and, optionally, a specific to-do item.	M
FR-2.3	If the user starts the timer without selecting a subject, the system shall fall back to an auto-created "Other" subject rather than blocking the start.	S
FR-2.4	The timer shall persist its state (phase, start timestamp, pause timestamp, accumulated pause time) to <code>localStorage</code> , and recompute remaining/elapsed time from timestamps — not an interval counter — so that switching tabs, backgrounding the app, or refreshing the page never causes drift or data loss.	M
FR-2.5	The user shall be able to pause, resume, and reset an in-progress session. Resetting a session with more than 0 elapsed seconds shall save it as an incomplete <code>study_session</code> ; resetting at exactly 0 seconds shall discard it.	M
FR-2.6	The user shall be able to log a "distraction" event during a focus phase via a dedicated button, incrementing a per-session distraction counter.	M
FR-2.7	When a focus phase completes, the system shall persist a completed <code>study_session</code> row (duration, distraction count) and automatically transition to a break phase. When a break phase completes, the timer shall return to idle.	M
FR-2.8	On phase completion, the system shall play an audible alert: a user-uploaded custom sound if one is configured, otherwise a generated beep repeated 7 times.	S
FR-2.9	The user shall be able to upload, replace, remove, and test-preview a custom completion sound, stored client-side (<code>IndexedDB</code>), max playback capped at 15 seconds.	S
FR-2.10	While a session is active, the system shall update a <code>study_presence</code> flag (studying / not studying) visible to friends, without exposing subject or todo content.	M

3.3 To-Do List (FR-3x)

ID	Requirement	Priority
FR-3.1	The user shall be able to create a to-do with a title, optional subject, optional linked exam, optional due date, and optional estimated minutes.	M
FR-3.2	The user shall be able to mark a to-do done/undone, edit its title, and delete it.	M
FR-3.3	The system shall compute and display each to-do's actual time spent, derived by summing linked <code>study_sessions.duration_seconds</code> (view <code>v_todo_time</code>) — never a manually entered value.	M
FR-3.4	The user shall be able to bulk-delete all completed to-dos in a single action ("clear completed").	S
FR-3.5	The user shall be able to convert an existing to-do into a recurring daily task (see FR-9.x), linking future auto-generated instances back to the same template.	S

3.4 Subjects (FR-4x)

ID	Requirement	Priority
FR-4.1	The user shall be able to create, rename, recolor, and archive ("remove") subjects.	M / S
FR-4.2	New subjects shall receive a color automatically assigned from a fixed palette if the user does not pick one, ensuring distinct colors across a user's subjects for chart legibility.	S
FR-4.3	The system shall provide/auto-create a special "Other" subject as the fallback target for timer sessions started without an explicit subject (FR-2.3).	S

3.5 Exams (FR-5x)

ID	Requirement	Priority
FR-5.1	The user shall be able to register an exam with a title, subject, and exam date, and view a computed D-day countdown.	M
FR-5.2	When creating a new exam, the system shall surface the user's past retrospectives for the same subject (up to 5, most recent first) to support backward planning.	M
FR-5.3	The user shall be able to attach a checklist / to-dos to an exam (via <code>todos.exam_id</code>) for backward-planning study tasks.	M
FR-5.4	After an exam date, the user shall be able to record a retrospective: a numeric score and free-text reflection, timestamped on save.	M
FR-5.5	The user shall be able to delete an exam.	M

3.6 Past-Question Tracking & Spaced Repetition (FR-6x)

ID	Requirement	Priority
FR-6.1	The user shall be able to log a past exam question with a title, optional source, optional memo, optional subject, and optional linked exam.	M
FR-6.2	The user shall be able to grade a question as O (correct), X (incorrect), or "unsure" (헷갈림) each time they review it.	M
FR-6.3	On each grading, the system shall automatically recompute the question's next review interval: O advances the interval $1 \rightarrow 3 \rightarrow 7 \rightarrow 14$ days (capping at 14); X or unsure resets the interval to 1 day. <code>next_review_date</code> and <code>review_count</code> update accordingly. This logic runs entirely in a Postgres trigger (<code>apply_question_review</code>), not client code.	M
FR-6.4	The system shall provide a "wrong answers" view listing all questions whose most recent result is X or unsure, ordered by next review date.	M
FR-6.5	The user shall be able to delete a question.	M

3.7 Streaks (FR-7x)

ID	Requirement	Priority
FR-7.1	The system shall maintain a current streak and longest streak per user, counted in consecutive days where total study time meets a configurable daily goal (default 25 minutes).	M
FR-7.2	The system shall grant a configurable number of "rest passes" per ISO week (default 1) that automatically preserve the streak for a day that missed the goal, consumed oldest-shortfall-day-first when the streak is recalculated.	M
FR-7.3	Rest-pass consumption shall never be back-applied to dates before the user's first recorded study day, preventing an unused account from silently accumulating an artificial streak.	M
FR-7.4	Streak recalculation shall be triggered automatically by any insert/update/delete on <code>study_sessions</code> (via <code>daily_stats</code> sync), not by a scheduled job or client call.	M

3.8 Stats (FR-8x)

ID	Requirement	Priority
FR-8.1	The system shall display a daily/weekly totals bar chart of study time.	M
FR-8.2	The system shall display a time-by-subject breakdown (last 30 days) as a chart, rendering each subject in its own distinct color (from FR-4.2).	M
FR-8.3	The system shall display a distraction-count trend over time.	M
FR-8.4	The system shall display the user's current/longest streak.	M

3.9 Friends & Privacy (FR-9x)

ID	Requirement	Priority
FR-9.1	Each user shall have a unique, shareable friend code (auto-generated at signup).	M
FR-9.2	A user shall be able to send a friend request by entering another user's friend code, cancel/resend after rejection, and accept or decline an incoming request. Friendship is created as two mirrored rows (bidirectional) only after acceptance.	M
FR-9.3	A user shall be able to remove an existing friend.	M
FR-9.4	Friends shall be able to see only : today's total study seconds, this week's total study seconds, current streak, and live "currently studying" presence. Friends shall never be able to see subject names, to-do titles, exam names, or question content, under any circumstance — this shall be structurally enforced by PostgreSQL Row Level Security (the tables exposed to friends contain no such columns at all), not solely by frontend filtering.	M
FR-9.5	The system shall rank/list friends by weekly study time and by weekly distraction count.	S
FR-9.6	A user shall be able to opt in/out of sharing their weekly manually-entered screen-time minutes with friends via a per-user setting (<code>share_screen_time</code>), defaulting to off. This visibility shall also be enforced at the RLS layer, via a <code>SECURITY DEFINER</code> helper function so the friend's own setting is readable without exposing the settings table itself.	S

3.10 Daily Planner (FR-10x)

ID	Requirement	Priority
FR-10.1	The user shall be able to view their to-dos in Day, Week, Month, and Year views.	S
FR-10.2	The system shall render a completion-percentage calendar (color intensity by % of that day's due to-dos completed) for Month and Year views, computed directly from existing <code>todos.due_date / is_done</code> — no separate planner table.	S
FR-10.3	The user shall be able to define "recurring tasks" (a title, optional subject, optional estimated minutes) that function as daily habits.	S
FR-10.4	The system shall auto-generate a to-do instance for each active recurring task for a given date the first time that date is viewed, but never retroactively for dates before the recurring task was created.	S
FR-10.5	The user shall be able to toggle a recurring task's completion for a given day directly from a habit-tracker grid; toggling creates the day's to-do instance on demand if it does not yet exist.	S
FR-10.6	The user shall be able to rename a recurring task and activate/deactivate it; deactivating stops future auto-generation but leaves past instances intact (<code>recurring_task_id</code> uses <code>on delete set null</code> , so deleting the template does not delete history).	S
FR-10.7	Deleting a recurring task shall prompt the user for confirmation before proceeding.	S
FR-10.8	The user shall be able to write one free-text daily journal note per calendar date.	S

3.11 Settings (FR-11x)

ID	Requirement	Priority
FR-11.1	The user shall be able to configure focus/break minutes, long-break minutes, sessions until long break, daily streak goal minutes, and rest passes per week.	M
FR-11.2	Numeric input fields shall accept and display an empty state while typing, coercing to a valid number (falling back to the default) only on submit — preventing leading-zero artifacts (e.g. typing into a cleared field never producing "025").	S
FR-11.3	The user shall be able to manage subjects (FR-4.x) and the custom timer sound (FR-2.9) from Settings.	S

3.12 Data Backup (FR-12x)

ID	Requirement	Priority
FR-12.1	The user shall be able to export all of their own data (subjects, exams, todos, study sessions, questions, question reviews, screen-time entries, user settings) as a single downloadable JSON file.	M
FR-12.2	Derived/aggregate tables (<code>daily_stats</code> , <code>streaks</code> , <code>rest_passes</code>) shall be excluded from export/import, since they are fully reconstructible from <code>study_sessions</code> by existing triggers once sessions are restored.	M
FR-12.3	The user shall be able to import a previously exported JSON file. Import shall upsert rows under the currently signed-in user's ID regardless of the <code>user_id</code> values in the file, so that re-importing the same file is safe and importing another account's file cannot write data under a different owner.	M
FR-12.4	Import shall be rejected with a clear error if the file does not look like a StudyPlanner backup (structural validation).	M

3.13 Installability (FR-13x)

ID	Requirement	Priority
FR-13.1	The system shall be installable as a Progressive Web App (manifest, icons, service worker) on desktop and mobile home screens.	S
FR-13.2	The service worker shall not cache application assets, so that a new deploy is always reflected immediately rather than serving a stale cached build.	S

4. Use Cases

UC-1 — Start a Focus Timer Session

- **Actor:** Student
- **Precondition:** User is signed in.
- **Trigger:** User navigates to the Timer page and presses Start.
- **Main flow:**
 1. User optionally selects a subject and/or a to-do to link.
 2. User presses Start.
 3. If no subject was selected, the system resolves/creates the "Other" subject and uses it (FR-2.3).
 4. System calls `startSession()`, inserting a `study_sessions` row with `started_at = now()`, `study_date = today(local)`, and sets `study_presence.is_studying = true`.
 5. System persists timer state to `localStorage` and begins the focus countdown.

- **Postcondition:** A study session is in progress; friends who have this user can see `is_studying = true` if they check the Friends page.
- **Alternate flow:** User backgrounds the tab or refreshes the page — on return, the system reads `localStorage`, recomputes elapsed time from `phaseStartedAt`, and resumes the display exactly where it should be (FR-2.4).

UC-2 — Complete a Pomodoro Focus Phase

- **Actor:** Student, System
- **Precondition:** A focus-phase timer is running.
- **Trigger:** Elapsed time reaches the configured focus duration.
- **Main flow:**
 1. System plays the completion alert (custom sound or 7x beep).
 2. System calls `finishSession()` with the elapsed duration, distraction count, and `is_completed = true`.
 3. A Postgres trigger (`trg_study_sessions_stats`) recomputes `daily_stats` for that date, and recalculates the user's streak (FR-7.4).
 4. System clears `study_presence.is_studying`.
 5. System transitions the local timer into the break phase automatically.
- **Postcondition:** A completed session is recorded; today's stats, streak, and any to-do's actual time are all updated without further user action.

UC-3 — Log a Distraction

- **Actor:** Student
- **Precondition:** A focus phase is running.
- **Trigger:** User presses "Got distracted."
- **Main flow:** System increments the in-memory/persisted `distractionCount` for the current session. The count is saved to the `study_sessions` row when the session finishes (UC-2) or is reset (UC-1 alternate).
- **Postcondition:** Distraction count is reflected in Stats (FR-8.3) and, in aggregate, is visible to friends via weekly distraction ranking (FR-9.5).

UC-4 — Track a To-Do Against Actual Time Spent

- **Actor:** Student
- **Precondition:** A to-do exists with an estimated time.
- **Main flow:**
 1. User starts one or more timer sessions linked to that to-do (UC-1), possibly across multiple days.
 2. User views the to-do list.
 3. System computes `actual_seconds` for the to-do by summing all linked completed sessions (`v_todo_time` view) and displays it next to the estimate.
- **Postcondition:** User can compare estimate vs. reality without having manually logged any time themselves.

UC-5 — Convert a To-Do into a Recurring Habit

- **Actor:** Student
- **Precondition:** A to-do exists.
- **Main flow:**
 1. User chooses "make recurring" on a to-do.
 2. System creates a `recurring_tasks` row from the to-do's title/subject/estimate and links the existing to-do to it via `recurring_task_id`.

- **Postcondition:** From the next time the Planner is opened for a given date on/after the template's creation date, an instance to-do is auto-created for that date if missing (FR-10.4), and the habit appears in the habit-tracker grid.

UC-6 — Toggle a Habit-Tracker Cell

- **Actor:** Student
- **Precondition:** A recurring task exists.
- **Main flow:**
 1. User clicks a day cell for a recurring task in the habit grid.
 2. If no to-do instance exists yet for that (task, date) pair, the system creates one, marked done, in a single step.
 3. If an instance already exists, the system flips its `is_done` flag.
- **Postcondition:** The completion calendar and habit percentage update immediately (react-query invalidation).

UC-7 — Backward-Plan for an Exam

- **Actor:** Student
- **Precondition:** User is signed in.
- **Main flow:**
 1. User creates an exam with a title, subject, and date.
 2. System shows the user's last 5 retrospectives for that subject, if any, to inform planning.
 3. User adds a checklist of to-dos linked to the exam (`todos.exam_id`), each with its own due date working backward from the exam date.
 4. User studies via the Timer, optionally linking sessions to those to-dos.
- **Postcondition:** Exam detail page shows countdown, checklist completion, and, once sessions are logged, actual prep time per checklist item.

UC-8 — Record an Exam Retrospective

- **Actor:** Student
- **Precondition:** An exam exists (typically past its date, though not enforced).
- **Main flow:** User opens the exam, enters a score and free-text retrospective, and saves. System stamps `retrospective_at = now()` and updates the exam row.
- **Postcondition:** This retrospective becomes visible the next time the user creates a new exam for the same subject (UC-7 step 2), closing the planning loop.

UC-9 — Review a Past Question with Spaced Repetition

- **Actor:** Student, System
- **Precondition:** A question exists (created with `interval_days = 1`, `next_review_date = today`).
- **Main flow:**
 1. User opens the Wrong Answers / review view, which lists questions due today or overdue (`next_review_date <= today`, last result X or unsure) — plus any question up for its scheduled review.
 2. User attempts the question and grades it O / X / unsure.
 3. System inserts a `question_reviews` row.
 4. Trigger `apply_question_review` fires: if the grade is O, the interval advances (1→3→7→14); if X or unsure, the interval resets to 1 and `next_review_date = today + 1`.
- **Postcondition:** The question resurfaces automatically on its new due date with no manual scheduling by the user.

UC-10 — Maintain a Study Streak Using a Rest Pass

- **Actor:** Student, System
- **Precondition:** User has an active streak and has studied on at least one prior day.
- **Main flow:**
 1. User does not meet the daily goal on a given day (e.g., misses studying entirely).
 2. On the next `study_sessions` change (any day), the system recalculates the streak by walking backward from today.
 3. When it reaches the shortfall day, it checks whether a rest pass is still available for that ISO week (`rest_passes_per_week` minus already-used passes that week).
 4. If available, the system consumes one rest pass, marks that day's `daily_stats.rest_pass_used = true`, counts the day toward the streak, and continues walking backward. If not available, the streak count stops at that day.
- **Postcondition:** The user's streak is preserved across one missed day per week (by default) without requiring any manual action.

UC-11 — Add and Accept a Friend

- **Actor:** Student A, Student B
- **Precondition:** Both users have accounts; A knows B's friend code.
- **Main flow:**
 1. A submits B's friend code via `send_friend_request` RPC.
 2. System validates the code exists, isn't A's own, and that A/B aren't already friends; inserts a `friend_requests` row (or revives a previously rejected one).
 3. B sees the incoming request and accepts via `respond_friend_request(accept = true)`.
 4. System inserts two mirrored `friendships` rows (A→B and B→A) inside one transaction.
- **Postcondition:** A and B can now see each other's shared aggregates (UC-12). Either can later call `remove_friend` to delete both mirrored rows.

UC-12 — View a Friend's Shared Stats (Privacy-Preserving)

- **Actor:** Student
- **Precondition:** Viewer and target are friends.
- **Main flow:**
 1. Viewer opens the Friends page.
 2. System calls `get_friends_summary()`, which — running with the *viewer's own* RLS context (`security invoker`) — joins `daily_stats`, `streaks`, and `study_presence` for each friend.
 3. Because those tables' RLS policies only allow rows where `user_id = auth.uid()` OR `are_friends(auth.uid(), user_id)`, the query naturally returns rows only for confirmed friends, with only the numeric columns those tables contain.
- **Postcondition:** Viewer sees each friend's today/week seconds, current streak, and live studying status. There is no code path — even a compromised or buggy frontend — through which the viewer could retrieve a friend's subject names, to-dos, exam titles, or question content, because those tables have no "friend" RLS policy at all (FR-9.4).

UC-13 — Share Weekly Screen Time with Friends

- **Actor:** Student
- **Precondition:** User has entered a weekly screen-time value.
- **Main flow:**
 1. User enters their screen-time minutes for the current week (manual input; not tracked automatically).

2. User toggles "share with friends" on in Settings (`user_settings.share_screen_time`).
 3. A friend viewing the ranking triggers a `screen_time_entries` select; RLS policy calls `shares_screen_time(owner_id)` (a `SECURITY DEFINER` function) to check the owner's setting without needing direct read access to `user_settings`.
- **Postcondition:** The entry is visible to friends only while the toggle is on; turning it off makes the row disappear from friends' views on the next query, with no caching of a stale "shared" state.

UC-14 — Export and Re-Import All Data

- **Actor:** Student
- **Precondition:** User is signed in.
- **Main flow (export):** User clicks Export in Settings; system fetches all owned rows across the core tables in parallel and triggers a browser download of a timestamped JSON file.
- **Main flow (import):** User selects a previously exported (or another device's) JSON file; system validates its shape, then upserts each table's rows in FK-safe order, overwriting the file's `user_id` values with the currently signed-in user's ID.
- **Postcondition:** All directly-recorded data is restored; `daily_stats`, `streaks`, and `rest_passes` are not imported but are automatically rebuilt by the existing triggers as soon as `study_sessions` rows land (FR-12.2).
- **Exception:** If the file is missing/malformed (no `subjects` array), the system rejects the import with an explicit error before writing anything.

UC-15 — Install StudyPlanner as a Phone App

- **Actor:** Student
- **Precondition:** User visits the deployed site in a PWA-capable mobile browser.
- **Main flow:** Browser detects the manifest and offers an "Add to Home Screen" / install prompt; user accepts; an app icon is placed on the device home screen.
- **Postcondition:** Subsequent launches open StudyPlanner in a standalone window (no browser chrome), without requiring the user to type the URL again. Because the service worker performs no caching, the next deploy is picked up on the next launch automatically.

5. Data Model Summary

```
profiles(id, display_name, friend_code, timezone)
user_settings(user_id, focus_minutes, break_minutes, long_break_minutes,
              sessions_until_long_break, streak_goal_minutes,
              rest_passes_per_week, share_screen_time)

subjects(id, user_id, name, color, archived)
exams(id, user_id, subject_id, title, exam_date, score, retrospective, retrospective_at)
todos(id, user_id, subject_id, exam_id, recurring_task_id, title,
       estimated_minutes, due_date, is_done, completed_at, sort_order)
recurring_tasks(id, user_id, subject_id, title, estimated_minutes, active)
daily_notes(user_id, note_date, content) -- PK (user_id, note_date)

study_sessions(id, user_id, subject_id, todo_id, started_at, ended_at,
               duration_seconds, distraction_count, study_date, is_completed)
-- source of truth; all aggregates below are derived from this table by triggers

questions(id, user_id, subject_id, exam_id, title, source, memo,
           last_result, interval_days, next_review_date, review_count)
question_reviews(id, user_id, question_id, result, reviewed_at)

daily_stats(user_id, study_date, total_seconds, session_count,
            distraction_count, rest_pass_used) -- derived, friend-visible
streaks(user_id, current_streak, longest_streak, last_study_date) -- derived, friend-visible
rest_passes(user_id, used_date, week_start) -- derived
study_presence(user_id, is_studying, session_started_at) -- friend-visible
```

```

screen_time_entries(user_id, week_start, minutes) -- friend-visible only if shared
friend_requests(id, requester_id, addressee_id, status)
friendships(user_id, friend_id) -- mirrored pair per friendship

-- views (security_invoker: RLS still applies through the view)
v_todo_time(todo_id, actual_seconds)
v_subject_time(subject_id, study_date, seconds)

```

Key server-side functions:

Function	Purpose
handle_new_user()	Trigger on auth.users insert — bootstraps profile/settings/streak/presence rows.
sync_daily_stats_for(user, date) / handle_study_session_change()	Trigger — keeps daily_stats in sync with study_sessions and calls recalculate_streak.
recalculate_streak(user)	Walks backward from today applying the goal/rest-pass rules (FR-7.x).
apply_question_review()	Trigger — applies the spaced-repetition interval rules (FR-6.3).
are_friends(a, b)	SECURITY DEFINER — used inside RLS policies to check friendship without recursive RLS.
shares_screen_time(user)	SECURITY DEFINER — lets the RLS policy check a friend's sharing toggle without direct table access.
send_friend_request / respond_friend_request / remove_friend	SECURITY DEFINER RPCs — the only way friend_requests/friendships rows can be written (no direct INSERT policy exists).
get_friends_summary()	SECURITY INVOKER — aggregates friends' numbers, relying on the caller's own RLS context.

6. Non-Functional Requirements

ID	Requirement
NFR-1	Privacy by construction: Any table readable by a friend shall contain no subject/todo/exam/question content columns, so a frontend bug cannot leak that data (FR-9.4).
NFR-2	Timer accuracy: Elapsed/remaining time shall be computed from timestamps, not accumulated tick counts, so background tabs and refreshes never desynchronize the timer.
NFR-3	No silent failures: All data-mutating forms shall surface errors from failed Supabase calls to the user via visible UI state, not swallow them.
NFR-4	Schema changes require explicit confirmation: Any migration must be explained to the user and confirmed before being applied against the production database.
NFR-5	Fresh deploys: The PWA service worker shall not cache build assets, so every new deploy is served immediately rather than a stale cached version.

7. Glossary

Term	Meaning
D-day	Countdown/count-up display of days until (or since) an exam date.
Rest pass	A once-per-week (configurable) automatic streak protection for a day that missed the study goal.

Term	Meaning
Spaced repetition	The 1→3→7→14-day review-interval scheduling applied to past-exam questions.
Recurring task	A daily habit template that auto-generates a to-do instance per day.
Friend code	An 8-character unique code used to find and add another user as a friend.